System Design

Day 22

Event Versioning, Schema Evolution, and Upcasting

Keeping old and new events usable together as systems evolve.

1 IT COVERS

- event schema versioning fundamentals
- backward and forward compatibility
- upcaster pattern for legacy event migration
- replay safety with mixed-version streams
- defaulting and transformation strategies
- trade-offs in long-lived event stores

2 CORE IDEAS

1. Why version events
Event contracts evolve, but historical events must stay replayable.

2. Compatibility
New consumers should handle old events; old consumers may not understand new fields.

3. Upcasting
Transform older event versions into a canonical latest shape at read time.

4. Replay safety
Projectors should operate on canonical events; replay tests are critical.

5. Trade-offs
Upcasting avoids rewriting history, but adds runtime transformation cost and version debt.

3 DAY-22 SAMPLE

Event Upcasting Ledger

Legacy service (v1 event) → EventStore
Current service (v2 event) → EventStore

EventStore
Append-only log of events (mixed versions)

→ **EventUpcaster**
Converts older versions to latest canonical shape

→ **Canonical events**
Latest schema used by all projectors

→ **AccountProjector**
Applies canonical events in order

→ **Rebuilt balance**
Final aggregate state from replay

DATA FLOW

1. Mixed-version events are appended to the stream
2. Rebuild reads raw events
3. Upcaster converts old versions to latest form
4. Projector applies canonical events in order
5. Final aggregate state is returned from replay

INTERVIEW TAKEAWAYS

- Evolution strategy should be explicit from day one
- Upcasting preserves immutable history while code evolves
- Canonical event handling simplifies projector correctness
- Replay validation is mandatory after schema changes
- Version debt grows, so pruning and compaction matter

5 NEXT IMPROVEMENTS

- multi-step upcasting chains (v1 -> v2 –> v3)
- schema registry compatibility checks
- snapshot-based rebuild acceleration
- projection contract tests with golden fixtures
- metrics for upcast hit rate and replay latency

Design for change. Upcasting lets you evolve today without losing your past.

Keep learning. Keep building.

System Design - Day 22

Event Versioning, Schema Evolution, and Upcasting

Prepared from Day-22 notes and sample implementation

Overview from README.md

Day 22: Event Versioning, Schema Evolution, and Upcasting

Day 22 focuses on how event-driven systems evolve event schemas safely without breaking replay, projections, or consumer compatibility.

It covers:
- event schema versioning fundamentals
- backward and forward compatibility concerns
- upcaster pattern for legacy event migration
- replay safety with mixed-version event streams
- defaulting and transformation strategies
- operational trade-offs in long-lived event stores

In short, Day 22 is about keeping old and new events usable together as systems evolve.

Core ideas

Why version events
- Event contracts change as product requirements evolve.
- Historical events remain in storage and must still be replayable.
- Versioning prevents silent corruption when payload shapes diverge.

Compatibility concerns
- Older consumers may not understand new fields or formats.
- New consumers must still process legacy events in existing streams.
- Clear compatibility policy is required per event type.

Upcasting pattern
- `Upcasting` transforms older event versions into a canonical latest shape at read time.
- Instead of rewriting the full event store, transforms are applied during replay.
- This keeps storage immutable while enabling code evolution.

Replay and projection safety
- Projectors should operate on canonical event shapes.
- Missing upcasters or inconsistent defaults can break deterministic rebuilds.
- Replay tests are critical after schema changes.

Trade-offs
- Upcasting reduces migration risk but adds runtime transformation cost.
- Version chains can grow and become harder to reason about.
- Periodic compaction or snapshotting can reduce long-term replay overhead.

Day-22 sample: Event Upcasting Ledger

This repository includes a small event-versioning sample in both Python and Java.

Functional requirements
- Store events with explicit schema versions
- Keep legacy (`v1`) and latest (`v2`) events in the same stream
- Upcast old events into canonical latest form during read/replay

- Rebuild account balance from mixed-version history
- Fail fast for unsupported version transitions

High-level components
- `EventStore`: append-only event storage
- `EventUpcaster`: transforms old versions to latest
- `AccountProjector`: applies canonical events to rebuild state
- `CommandService`: emits latest-version events

Data flow
1. Legacy and current services append events with their own schema versions
2. Rebuild reads raw events from the stream
3. Upcaster converts each event to latest supported version
4. Projector applies canonical events in sequence
5. Final aggregate state is returned from replay

Project structure

```
Day-22/
README.md
event-upcasting-sample/
python/
event_upcasting_ledger.py
demo.py
java/
EventUpcastingLedger.java
Main.java
```

Run the sample

Python

```
cd event-upcasting-sample\python
python demo.py
```

Java

```
cd event-upcasting-sample\java
javac Main.java EventUpcastingLedger.java
java Main
```

Interview takeaways
- Event evolution strategy should be explicit from day one in event-sourced systems
- Upcasting is a common way to preserve immutable history with evolving schemas
- Canonical event handling simplifies projector correctness and testing
- Replay validation is a mandatory guardrail before deploying schema changes
- Version debt accumulates, so pruning/compaction strategy matters long term

Next improvements
- Add multi-step upcasting chains (v1 -> v2 -> v3)
- Add schema registry compatibility checks for producers
- Add snapshot-based rebuild acceleration after heavy upcasting
- Add projection contract tests with golden historical fixtures
- Add metrics for upcast hit rate and replay latency

Reference Implementation

Python and Java sample source files

File: event-upcasting-sample/python/demo.py

```python
from event_upcasting_ledger import (
    CommandService,
    EventStore,
    EventUpcaster,
    rebuild_balance,
)


def main() -> None:
    store = EventStore()
    upcaster = EventUpcaster()
    commands = CommandService(store)
    account_id = "acct-2200"

    # Simulate a legacy service that produced v1 events without currency.
    store.append(
        account_id=account_id,
        event_type="FundsDeposited",
        schema_version=1,
        payload={"amount_cents": 10_000},
    )
    store.append(
        account_id=account_id,
        event_type="FundsWithdrawn",
        schema_version=1,
        payload={"amount_cents": 2_000},
    )

    # Current service writes latest v2 schema.
    commands.deposit(account_id, 1_500, currency="USD")
    commands.withdraw(account_id, 700, currency="USD")

    print("raw events")
    for event in store.events_for_account(account_id):
        print(
            f"  seq={event.sequence} type={event.event_type} "
            f"v={event.schema_version} payload={event.payload}"
        )

    print("\ncanonical events after upcasting")
    for event in store.events_for_account(account_id):
        canonical = upcaster.upcast(event)
        print(
            f"  seq={canonical.sequence} type={canonical.event_type} "
            f"v={canonical.schema_version} amount={canonical.amount_cents} "
            f"currency={canonical.currency}"
        )

    balance = rebuild_balance(account_id, store, upcaster)
    print(f"\nrebuilt balance: {balance}")


if __name__ == "__main__":
    main()
```

File: event-upcasting-sample/python/event_upcasting_ledger.py

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Any


@dataclass(frozen=True)
class RawEvent:
    sequence: int
    account_id: str
    event_type: str
    schema_version: int
    payload: dict[str, Any]


@dataclass(frozen=True)
class CanonicalEvent:
    sequence: int
    account_id: str
    event_type: str
    schema_version: int
    amount_cents: int
    currency: str


class EventStore:
    def __init__(self) -> None:
        self._events: list[RawEvent] = []
        self._next_sequence = 1

    def append(
        self,
        account_id: str,
        event_type: str,
        schema_version: int,
        payload: dict[str, Any],
    ) -> RawEvent:
        if not account_id:
            raise ValueError("account_id must be non-empty")
        if not event_type:
            raise ValueError("event_type must be non-empty")
        if schema_version <= 0:
            raise ValueError("schema_version must be positive")

        event = RawEvent(
            sequence=self._next_sequence,
            account_id=account_id,
            event_type=event_type,
            schema_version=schema_version,
            payload=payload,
        )
        self._events.append(event)
        self._next_sequence += 1
        return event

    def events_for_account(self, account_id: str) -> list[RawEvent]:
        return [event for event in self._events if event.account_id == account_id]


class EventUpcaster:
    _LATEST_VERSION_BY_TYPE = {
        "FundsDeposited": 2,
        "FundsWithdrawn": 2,
    }

    def upcast(self, event: RawEvent) -> CanonicalEvent:
        latest = self._LATEST_VERSION_BY_TYPE.get(event.event_type)
        if latest is None:
            raise ValueError(f"unsupported event_type: {event.event_type}")
        if event.schema_version > latest:
            raise ValueError(
                f"unsupported future schema for {event.event_type}: {event.schema_version} > {latest}"
```

```
            )

        current_version = event.schema_version
        payload = dict(event.payload)

        while current_version < latest:
            payload = self._upcast_once(event.event_type, current_version, payload)
            current_version += 1

        amount_cents = int(payload["amount_cents"])
        currency = str(payload["currency"])
        return CanonicalEvent(
            sequence=event.sequence,
            account_id=event.account_id,
            event_type=event.event_type,
            schema_version=current_version,
            amount_cents=amount_cents,
            currency=currency,
        )

    @staticmethod
    def _upcast_once(
        event_type: str, from_version: int, payload: dict[str, Any]
    ) -> dict[str, Any]:
        if from_version == 1 and event_type in {"FundsDeposited", "FundsWithdrawn"}:
            return {
                "amount_cents": int(payload["amount_cents"]),
                "currency": "USD",
            }
        raise ValueError(f"missing upcaster for {event_type} v{from_version}")


class AccountProjector:
    def __init__(self) -> None:
        self.balance_cents = 0

    def apply(self, event: CanonicalEvent) -> None:
        if event.currency != "USD":
            raise ValueError(f"unsupported currency: {event.currency}")

        if event.event_type == "FundsDeposited":
            self.balance_cents += event.amount_cents
            return
        if event.event_type == "FundsWithdrawn":
            self.balance_cents -= event.amount_cents
            return
        raise ValueError(f"unsupported event_type: {event.event_type}")


class CommandService:
    _LATEST_VERSION = 2

    def __init__(self, event_store: EventStore) -> None:
        self._event_store = event_store

    def deposit(self, account_id: str, amount_cents: int, currency: str = "USD") -> None:
        if amount_cents <= 0:
            raise ValueError("amount_cents must be positive")
        self._event_store.append(
            account_id=account_id,
            event_type="FundsDeposited",
            schema_version=self._LATEST_VERSION,
            payload={"amount_cents": amount_cents, "currency": currency},
        )

    def withdraw(self, account_id: str, amount_cents: int, currency: str = "USD") -> None:
        if amount_cents <= 0:
            raise ValueError("amount_cents must be positive")
        self._event_store.append(
            account_id=account_id,
            event_type="FundsWithdrawn",
            schema_version=self._LATEST_VERSION,
            payload={"amount_cents": amount_cents, "currency": currency},
```

```
        )


def rebuild_balance(account_id: str, store: EventStore, upcaster: EventUpcaster) -> int:
    projector = AccountProjector()
    for raw_event in store.events_for_account(account_id):
        canonical = upcaster.upcast(raw_event)
        projector.apply(canonical)
    return projector.balance_cents
```

File: event-upcasting-sample/java/Main.java

```java
import java.util.LinkedHashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        EventUpcastingLedger.EventStore store = new EventUpcastingLedger.EventStore();
        EventUpcastingLedger.EventUpcaster upcaster = new EventUpcastingLedger.EventUpcaster();
        EventUpcastingLedger.CommandService commands = new EventUpcastingLedger.CommandService(store);
        String accountId = "acct-2200";

        // Simulate legacy v1 events without currency.
        Map<String, Object> v1Deposit = new LinkedHashMap<>();
        v1Deposit.put("amount_cents", 10_000);
        store.append(accountId, "FundsDeposited", 1, v1Deposit);

        Map<String, Object> v1Withdraw = new LinkedHashMap<>();
        v1Withdraw.put("amount_cents", 2_000);
        store.append(accountId, "FundsWithdrawn", 1, v1Withdraw);

        // Current service writes latest v2 schema.
        commands.deposit(accountId, 1_500, "USD");
        commands.withdraw(accountId, 700, "USD");

        System.out.println("raw events");
        for (EventUpcastingLedger.RawEvent event : store.eventsForAccount(accountId)) {
            System.out.println(
                "  seq="
                    + event.getSequence()
                    + " type="
                    + event.getEventType()
                    + " v="
                    + event.getSchemaVersion()
                    + " payload="
                    + event.getPayload()
            );
        }

        System.out.println("\ncanonical events after upcasting");
        for (EventUpcastingLedger.RawEvent event : store.eventsForAccount(accountId)) {
            EventUpcastingLedger.CanonicalEvent canonical = upcaster.upcast(event);
            System.out.println(
                "  seq="
                    + canonical.getSequence()
                    + " type="
                    + canonical.getEventType()
                    + " v="
                    + canonical.getSchemaVersion()
                    + " amount="
                    + canonical.getAmountCents()
                    + " currency="
                    + canonical.getCurrency()
            );
        }

        int balance = EventUpcastingLedger.rebuildBalance(accountId, store, upcaster);
        System.out.println("\nrebuilt balance: " + balance);
    }
}
```

File: event-upcasting-sample/java/EventUpcastingLedger.java

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class EventUpcastingLedger {
    public static final class RawEvent {
        private final int sequence;
        private final String accountId;
        private final String eventType;
        private final int schemaVersion;
        private final Map<String, Object> payload;

        public RawEvent(
            int sequence,
            String accountId,
            String eventType,
            int schemaVersion,
            Map<String, Object> payload
        ) {
            this.sequence = sequence;
            this.accountId = accountId;
            this.eventType = eventType;
            this.schemaVersion = schemaVersion;
            this.payload = payload;
        }

        public int getSequence() {
            return sequence;
        }

        public String getAccountId() {
            return accountId;
        }

        public String getEventType() {
            return eventType;
        }

        public int getSchemaVersion() {
            return schemaVersion;
        }

        public Map<String, Object> getPayload() {
            return payload;
        }
    }

    public static final class CanonicalEvent {
        private final int sequence;
        private final String accountId;
        private final String eventType;
        private final int schemaVersion;
        private final int amountCents;
        private final String currency;

        public CanonicalEvent(
            int sequence,
            String accountId,
            String eventType,
            int schemaVersion,
            int amountCents,
            String currency
        ) {
            this.sequence = sequence;
            this.accountId = accountId;
            this.eventType = eventType;
            this.schemaVersion = schemaVersion;
            this.amountCents = amountCents;
            this.currency = currency;
        }
        public int getSequence() {
```

```
        return sequence;
    }

    public String getAccountId() {
        return accountId;
    }

    public String getEventType() {
        return eventType;
    }

    public int getSchemaVersion() {
        return schemaVersion;
    }

    public int getAmountCents() {
        return amountCents;
    }

    public String getCurrency() {
        return currency;
    }
}

public static final class EventStore {
    private final List<RawEvent> events = new ArrayList<>();
    private int nextSequence = 1;

    public RawEvent append(
        String accountId,
        String eventType,
        int schemaVersion,
        Map<String, Object> payload
    ) {
        if (accountId == null || accountId.isBlank()) {
            throw new IllegalArgumentException("accountId must be non-empty");
        }
        if (eventType == null || eventType.isBlank()) {
            throw new IllegalArgumentException("eventType must be non-empty");
        }
        if (schemaVersion <= 0) {
            throw new IllegalArgumentException("schemaVersion must be positive");
        }

        RawEvent event = new RawEvent(
            nextSequence,
            accountId,
            eventType,
            schemaVersion,
            new LinkedHashMap<>(payload)
        );
        events.add(event);
        nextSequence += 1;
        return event;
    }

    public List<RawEvent> eventsForAccount(String accountId) {
        List<RawEvent> result = new ArrayList<>();
        for (RawEvent event : events) {
            if (accountId.equals(event.getAccountId())) {
                result.add(event);
            }
        }
        return result;
    }
}

public static final class EventUpcaster {
    private final Map<String, Integer> latestVersionByType = new LinkedHashMap<>();

    public EventUpcaster() {
        latestVersionByType.put("FundsDeposited", 2);
        latestVersionByType.put("FundsWithdrawn", 2);
```

```
    }

    public CanonicalEvent upcast(RawEvent event) {
        Integer latest = latestVersionByType.get(event.getEventType());
        if (latest == null) {
            throw new IllegalArgumentException("unsupported eventType: " + event.getEventType());
        }
        if (event.getSchemaVersion() > latest) {
            throw new IllegalArgumentException(
                "unsupported future schema for "
                    + event.getEventType()
                    + ": "
                    + event.getSchemaVersion()
                    + " > "
                    + latest
            );
        }

        int currentVersion = event.getSchemaVersion();
        Map<String, Object> payload = new LinkedHashMap<>(event.getPayload());

        while (currentVersion < latest) {
            payload = upcastOnce(event.getEventType(), currentVersion, payload);
            currentVersion += 1;
        }

        int amountCents = ((Number) payload.get("amount_cents")).intValue();
        String currency = payload.get("currency").toString();

        return new CanonicalEvent(
            event.getSequence(),
            event.getAccountId(),
            event.getEventType(),
            currentVersion,
            amountCents,
            currency
        );
    }

    private Map<String, Object> upcastOnce(
        String eventType,
        int fromVersion,
        Map<String, Object> payload
    ) {
        if (fromVersion == 1 && ("FundsDeposited".equals(eventType) || "FundsWithdrawn".equals(eventType)))
            Map<String, Object> upgraded = new LinkedHashMap<>();
            upgraded.put("amount_cents", ((Number) payload.get("amount_cents")).intValue());
            upgraded.put("currency", "USD");
            return upgraded;
        }
        throw new IllegalArgumentException("missing upcaster for " + eventType + " v" + fromVersion);
    }
}

public static final class AccountProjector {
    private int balanceCents = 0;

    public void apply(CanonicalEvent event) {
        if (!"USD".equals(event.getCurrency())) {
            throw new IllegalArgumentException("unsupported currency: " + event.getCurrency());
        }
        switch (event.getEventType()) {
            case "FundsDeposited":
                balanceCents += event.getAmountCents();
                return;
            case "FundsWithdrawn":
                balanceCents -= event.getAmountCents();
                return;
            default:
                throw new IllegalArgumentException("unsupported eventType: " + event.getEventType());
        }
    }
    public int getBalanceCents() {
```

```
            return balanceCents;
        }
    }

    public static final class CommandService {
        private static final int LATEST_VERSION = 2;
        private final EventStore store;

        public CommandService(EventStore store) {
            this.store = store;
        }

        public void deposit(String accountId, int amountCents, String currency) {
            if (amountCents <= 0) {
                throw new IllegalArgumentException("amountCents must be positive");
            }
            Map<String, Object> payload = new LinkedHashMap<>();
            payload.put("amount_cents", amountCents);
            payload.put("currency", currency);
            store.append(accountId, "FundsDeposited", LATEST_VERSION, payload);
        }

        public void withdraw(String accountId, int amountCents, String currency) {
            if (amountCents <= 0) {
                throw new IllegalArgumentException("amountCents must be positive");
            }
            Map<String, Object> payload = new LinkedHashMap<>();
            payload.put("amount_cents", amountCents);
            payload.put("currency", currency);
            store.append(accountId, "FundsWithdrawn", LATEST_VERSION, payload);
        }
    }

    public static int rebuildBalance(
        String accountId,
        EventStore store,
        EventUpcaster upcaster
    ) {
        AccountProjector projector = new AccountProjector();
        for (RawEvent rawEvent : store.eventsForAccount(accountId)) {
            CanonicalEvent canonical = upcaster.upcast(rawEvent);
            projector.apply(canonical);
        }
        return projector.getBalanceCents();
    }
}
```